

without the `-n` option. We can do this as follows:

```
[root]# rsync -azv data/ root@172.16.208.11:/opt/backup/
root@172.16.208.11's password:
```

When you execute the above command, it will generate the following output:

```
sending incremental file list
created directory /opt/backup
./
data-1.iso
data-2.iso
data-3.iso

sent 3,223,059,976 bytes  received 110 bytes
17,374,986.99 bytes/sec
total size is 3,221,225,472  speedup is 1.00
```

c) Synchronise files from the remote to the local server:

To synchronise files from the remote to the local machine, we need to specify a remote path in the source and local path in the destination, as shown below:

```
[root]# rsync -azv root@172.16.208.11:/opt/backup/ /opt/
backup/

root@172.16.208.11's password:
```

When you execute the above command, it will generate the following output:

```
receiving incremental file list
created directory /opt/backup
./
data-1.iso
data-2.iso
data-3.iso

sent 84 bytes  received 3,223,059,976 bytes
17,374,986.85 bytes/sec
total size is 3,221,225,472  speedup is 1.00
```

d) Use remote shell during transfer: rsync allows us to specify remote shell during file transfer. For example, we can use SSH for secure file transfer from the source to the destination. We can use the `-e` option for this, as follows:

```
[root]# rsync -azv -e test data/data-2.iso
root@172.16.208.11:/opt/backup
root@172.16.208.11's password:
```

When you execute the above command, it will generate the following output:

```
sending incremental file list

sent 51 bytes  received 12 bytes  18.00 bytes/sec
total size is 1,073,741,824  speedup is 17,043,521.02
```

e) Resume file transfer: Let us take a real world situation — you are copying a large file and the operation failed due to a network connectivity issue. One of the options is to start the copy operation all over again, but this is not an efficient method. Here, we can take advantage of rsync's delta transfer algorithm. We can use the `-P` option, as follows.

First, partially copy the file from the source to the destination, as destination:

```
[root]# scp data/data-2.iso root@172.16.208.11:/opt/
backup/
root@172.16.208.11's password:
data-2.iso          73% 752MB 64.8MB/s   00:04
ETA^C # Press ctrl + c to simulate failure scenario
```

Now let us resume the file transfer using the `-P` option:

```
[root]# rsync -azv -P --progress data/data-2.iso
root@172.16.208.11:/opt/backup/
```

When you execute the above command, it will generate the following output:

```
sending incremental file list
data-2.iso
1,073,741,824 100% 23.83MB/s   0:00:42 (xfr#1, to-
chk=0/1)

sent 222,542,063 bytes  received 204,307 bytes
4,499,926.67 bytes/sec
total size is 1,073,741,824  speedup is 4.82
```

In this beginner-friendly article we have discussed some practical examples for using rsync. However, rsync has many more capabilities. You can use this article as a starting point, and implement it in many different and useful ways. 

 **Narendra Kangralkar**

The author is a FOSS enthusiast and loves exploring anything related to open source.